

Answer key — finished scripts and challenge solutions.

Welcome back. In *Scratch A* you built scripts that run top to bottom, scripts that start on a key or a click, and a `forever` loop that animates the cat. In *Scratch B* the cat starts to *make choices*: it looks at the world with light-blue *Sensing* blocks and then decides what to do with the orange `if` and `if-else` blocks. Open scratch.mit.edu, click *Create*, and let us teach the cat to react.

The block groups you will use. Each block has a colour. The colour tells you which group it is in. Find the colour first, then find the block.

Motion	(blue)	Move the sprite: move, turn, go to, point towards, change x or y.
Looks	(purple)	Change how the sprite looks: say, costume, size, backdrop.
Sound	(pink)	Play a sound.
Events	(yellow)	Start a script (green flag, key, click, when I receive) and broadcast messages.
Control	(orange)	Choose when and how often blocks run: wait, forever, repeat, if, stop.
Sensing	(light blue)	Ask about the world: touching?, key pressed?, mouse, ask and answer.

Project 1: Chase the mouse

The cat follows your mouse pointer wherever it goes. A forever loop keeps pointing the cat at the pointer and stepping toward it, so it chases you around the stage.

Get ready. Open a new project. The cat starts in the middle.

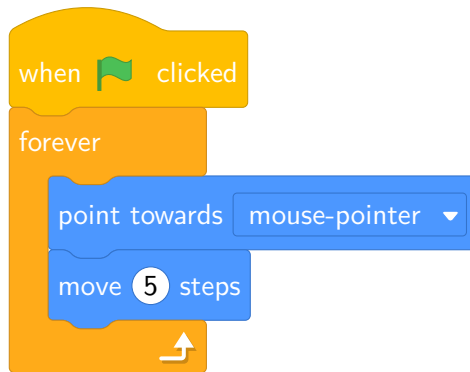
New blocks.

<code>point towards [mouse-pointer]</code>	A blue Motion block. It turns the sprite to face the pointer (or another sprite).
<code>move () steps</code>	Steps in the direction the sprite is now facing.

Build it.

1. Start a script with `when green flag clicked`.
2. From the orange Control blocks, drag in `forever`.
3. Inside the forever, add `point towards [mouse-pointer]` from the blue Motion blocks. Click the little arrow and choose **mouse-pointer**.
4. Under it, still inside the loop, add `move (5) steps`.
5. Click the green flag and move your mouse. The cat chases the pointer.

When you are done, your code should look like this:



The loop runs many times a second: each time it re-aims at the pointer and takes one small step, so the cat curves smoothly after your mouse.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Make the cat chase faster: change `move (5) steps` to a bigger number like 10.
Expected: Bigger steps make the cat catch up quicker.
2. Swap `move (5) steps` for `glide (0.3) secs to [mouse-pointer]` so the cat slides smoothly instead of stepping.
Expected: The cat glides toward the pointer each loop instead of jumping a step.
3. Make the cat speak when it reaches you: inside the loop add `if <distance to [mouse-pointer] < (30)>` then `say (Got you!) for (0.5) seconds`.
Expected: When the pointer is within 30 steps, the cat says Got you!

Project 2: Turn at the wall

The cat walks on its own. Every step it asks one question: am I touching the edge? If the answer is yes, it turns around and says ouch. This is your first `if` block - a choice the program makes.

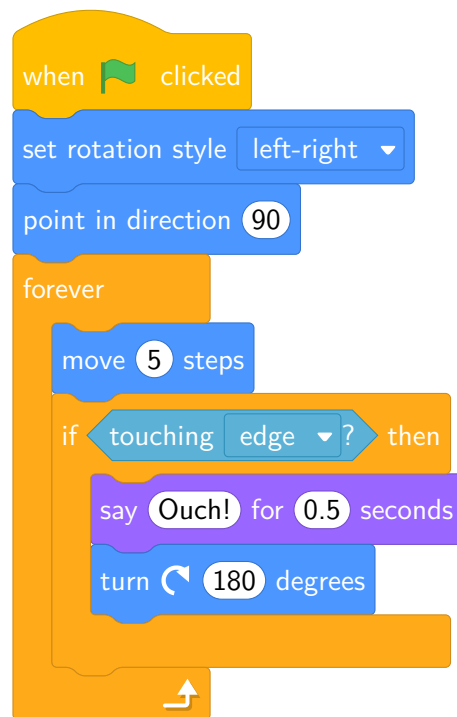
New blocks.

<code>if <> then</code>	An orange Control block. The blocks inside it run only when the hexagon question is true.
<code>touching [edge]?</code>	A light-blue Sensing hexagon. It is true when the sprite touches the thing you pick.

Build it.

1. Start with `when green flag clicked`, then `set rotation style [left-right]` and `point in direction (90)` so the cat faces right and never flips upside down.
2. Add a `forever` loop.
3. Inside it, add `move (5) steps`.
4. From the Control blocks, drag in `if <> then` under the move block.
5. Drop `touching [edge]?` (a Sensing hexagon) into the if's slot.
6. Inside the if, put `say (Ouch!) for (0.5) seconds` and `turn → (180) degrees`.
7. Click the green flag. The cat walks to a wall, says ouch, and turns back.

When you are done, your code should look like this:



A hexagon block is a question with a yes/no answer. It only fits in the pointy hexagon slot of an `if` (or another question), never on its own.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Add a sound on the bump: inside the `if`, before the turn, add `start sound [Boing]`.

Expected: The cat plays Boing each time it hits a wall.

2. Make it change pose on the bump too: add `next costume` inside the `if`.

Expected: The cat swaps costume every time it turns around.

3. Speed the cat up to bump more often: change `move (5) steps` to `move (10) steps`.

Expected: It reaches the walls faster and turns more often.

Project 3: The floor is lava

A red stripe on the stage is hot lava. The cat walks back and forth, but every step it chooses: if it is touching red, jump back to the start and shout; otherwise, keep walking. That is an `if-else` - two paths, and the program picks one.

Get ready. Click the *Stage* thumbnail (bottom right), then the *Backdrops* tab. Use the rectangle tool to paint a wide **red** stripe across the floor. That stripe is the lava.

New blocks.

`if <> then ... else ...`

Runs the first stack when the question is true, and the second (else) stack when it is false.

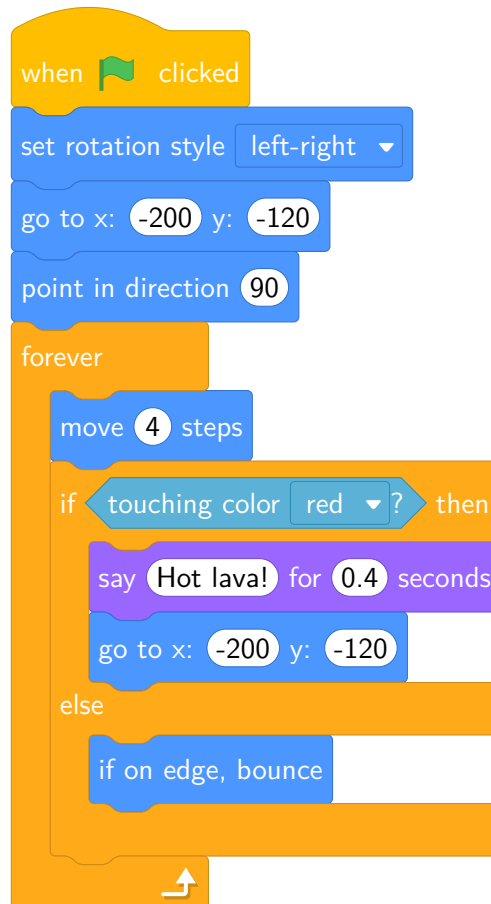
`touching color [ ]?`

A Sensing hexagon. True when the sprite touches that exact colour. Click the colour patch to pick red.

Build it.

1. Start with `when green flag clicked`, then `set rotation style [left-right]`, `go to x: -200 y: -120`, and `point in direction (90)`.
2. Add a `forever` loop, and inside it `move (4) steps`.
3. Under the move, drag in `if <> then ... else ...`.
4. Drop `touching color [red]?` into its slot. Click the colour patch on the block and pick the same red as your stripe.
5. In the `if (true)` part, put `say (Hot lava!) for (0.4) seconds` and `go to x: -200 y: -120`.
6. In the `else (false)` part, put `if on edge, bounce` so the cat turns around at the ends when it is safe.
7. Click the green flag. The cat paces the floor; step on red and it leaps back to the start.

When you are done, your code should look like this:



if runs its blocks only when the answer is yes. *if-else* always runs exactly one of its two stacks - never both, never neither.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Recolour the lava: paint your stripe a new colour, then click the patch in `touching color` and pick that colour to match.

Expected: The cat only resets on the colour you chose in the block.

2. Animate the safe walk: in the **else** part, add **next costume** so the legs move while the cat is safe.
Expected: The cat animates its walk except when it is on the lava.
 3. Make it harder: paint a second red stripe somewhere else. The same block already checks for red, so both stripes are now lava.
Expected: Touching either stripe sends the cat back to the start.
-

Project 4: Mini project: catch me if you can

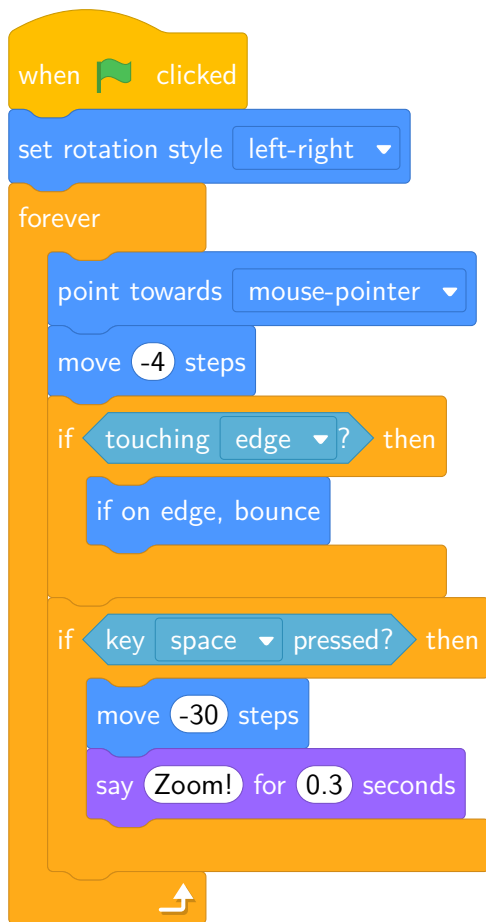
Put your new choices together into a little game. The cat runs away from your mouse, bounces off walls, and dashes when you press the space bar. Click it and it gives up. Three sensing questions, all in one forever loop.

Get ready. A new project, cat in the middle. You will build one main script and one little click script.
New blocks.

`key [space] pressed?` A Sensing hexagon. True while you hold that key down.

Build it.

1. Build the main script: `when green flag clicked`, then `set rotation style [left-right]`, then a `forever` loop.
 2. Inside the loop, add `point towards [mouse-pointer]` then `move (-4) steps`. A negative number moves the cat *away* from where it faces, so it flees the pointer.
 3. Still inside the loop, add `if <touching [edge]?>` then `if on edge, bounce`, so it does not get stuck on a wall.
 4. Add one more choice in the loop: `if <key [space] pressed?>` then `move (-30) steps` and `say (Zoom!) for (0.3) seconds` - a dash when you press space.
 5. Make the second script: `when this sprite clicked`, then `say (You caught me!) for (1) seconds` and `next costume`.
 6. Green flag. Chase the cat with your mouse, press space to make it dash, and click it to win.
- Your main script (left) and the click script (right):



Flee, bounce, and dash



Click: give up

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Make the cat harder to catch: change `move (-4) steps` to `move (-6) steps`.

Expected: It flees faster, so it is harder to corner.

2. Add a second dash key: in the loop add `if <key [up arrow] pressed?>` then `move (-30) steps`.

Expected: Now both space and the up arrow trigger a dash.

3. Warn the player near a wall: inside the `if <touching [edge]?>`, add `say (Wall!) for (0.3) seconds` before the bounce.

Expected: The cat shouts Wall! whenever it reaches the edge.